

implement A* algorithm for 8 puzzle game search problems.

Code :-

```
import heapq
```

```
goal = [[1,2,3],[4,5,6],[7,8,0]]
```

```
def h(s):
```

```
    return sum(abs(i-(v-1)//3) + abs(j-(v-1)%3)
```

```
               for i in range(3) for j in range(3)
```

```
               for v in [s[i][j]] if v != 0)
```

```
def next_states(s):
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if s[i][j] == 0:
```

```
                x, y = i, j
```

```
moves = [(0,1),(1,0),(0,-1),(-1,0)]
```

```
res = []
```

```
for dx, dy in moves:
```

```
    nx, ny = x + dx, y + dy
```

```
    if 0 <= nx < 3 and 0 <= ny < 3:
```

```
        t = [r[:] for r in s]
```

```
        t[x][y], t[nx][ny] = t[nx][ny], t[x][y]
```

```

        res.append(t)

    return res

def astar(s):
    pq = [(h(s), s)]
    vis = set()

    while pq:
        _, s = heapq.heappop(pq)

        if s == goal:
            for r in s:
                print(r)
            return

        vis.add(str(s))

        for n in next_states(s):
            if str(n) not in vis:
                heapq.heappush(pq, (h(n), n))

astar([[1,2,3],[4,0,6],[7,5,8]])

```

Theory :-

☒ Aim

To solve the 8-puzzle problem using the A* search algorithm.

☒ Theory

The 8-puzzle problem consists of a 3×3 grid with numbers from 1 to 8 and one empty space (0). The goal is to reach a target configuration by sliding tiles into the empty space.

The A* algorithm is an informed search algorithm that finds the optimal path to the goal using a heuristic function.

It uses the evaluation function:

$$f(n)=g(n)+h(n)$$

$$f(n)=g(n)+h(n)$$

Where:

- $g(n)$ = cost from start node to current node
- $h(n)$ = heuristic (estimated cost to reach goal) For the 8-puzzle, commonly used heuristic:
 - Manhattan Distance (sum of distances of tiles from their goal positions)

A* always selects the node with the lowest $f(n)$ value, making it both complete and optimal.

☒ Algorithm Steps

1. Start
2. Define initial state and goal state
3. Create a priority queue (OPEN list)
4. Insert initial state with its $f(n)$ value
5. Create a CLOSED list for visited states

6. Repeat until goal is reached:
 - Remove the state with lowest $f(n)$ from OPEN
 - If it is the goal state → Stop
 - Otherwise:
 - Generate all possible next states □
 - For each state:
 - Calculate $g(n)$, $h(n)$, and $f(n)$
 - If not visited, add to OPEN
 - Add current state to CLOSED
7. If OPEN becomes empty → No solution
8. Stop